

基于 MindSpore 的花卉识别 实验报告【严禁抄袭！】

by Kureha Shiro Dec.2025

一、实验目的

- 掌握卷积网络（CNN）相关基础知识点。
- 掌握使用 MindSpore 进行卷积神经网络开发的一般流程。
- 了解如何使用 MindSpore 进行花卉图片分类任务的训练和测试。
- 掌握使用 t-sne（T 分布随机近邻嵌入）实现对提取的特征进行可视化。

二、实验原理

2.1 功能结构

花卉识别实验总体设计如图 1 所示，该实验可以划分为数据处理、模型构建、图像识别三个主要的子实验。其中**数据处理子实验**包括数据读取、数据集划分、图像预处理三部分；**模型构建子实验**主要包括模型定义、模型训练以及模型部署三个部分；**图像识别子实验**内容主要包括模型准备、模型应用两部分。

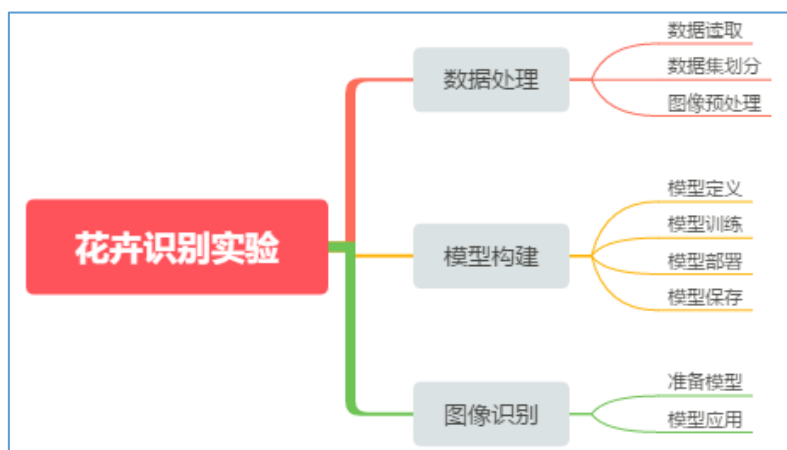


图 1 实验整体功能结构图

2.2 体系结构

按照体系结构划分，整个实验的体系结构可以划分为三部分，分别为**模型训练**、**模型保存**和**模型推理**，如图 2 所示。各层侧重点各不相同。

- **训练层**：运行在安装有 Mindspore 框架的服务器，配置计算加速卡。
- **推断层**：运行于开发环境，能够支持卷积神经网络的加速。
- **展示层**：运行于客户端应用程序，能够完成图像选择并实时显示推断层的计算结果。

各层之间存在单向依赖关系。推断层需要的网络模型由训练层提供，并根据需要进行必要的格式转换或加速重构。相应的，展示层要显示的元数据需要由推断层计算得到。

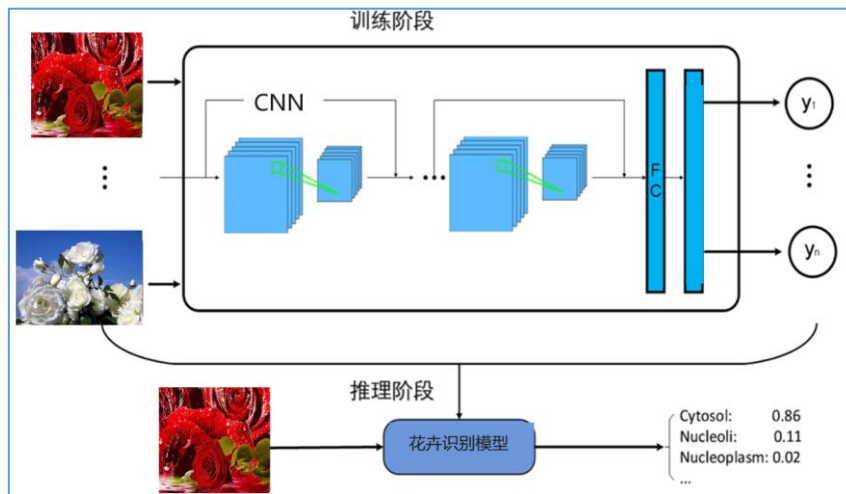


图 2 实验流程图

三、实验过程及结果

3.1 任务一

完成模型训练，粘上运行结果 Acc 与测试集个别图像的预测结果。

3.1.1 参数设定与训练过程可视化

首先修改 cfg 中的参数，批量大小 batch_size 设定为 **64**，训练轮数 epoch_size 设定为 **150**。而原始训练程序缺少可视化部分，于是定义了 LossAccCallback 类，用于训练过程中 Loss 和 Accuracy 的获取与曲线绘制（代码段 1）。

代码段 1：自定义 Callback，在每个 epoch 结束记录 loss 和 accuracy

```
from mindspore.train.callback import Callback
class LossAccCallback(Callback):
    def __init__(self, model, train_dataset, test_dataset):
        super(LossAccCallback, self).__init__()
        self.model = model
        self.train_dataset = train_dataset
        self.test_dataset = test_dataset
        self.train_loss = []
        self.test_acc = []

    def epoch_end(self, run_context):
        cb_params = run_context.original_args()
        loss = cb_params.net_outputs.asnumpy()
        self.train_loss.append(loss)

        metric = self.model.eval(self.test_dataset, dataset_sink_mode=False)
        self.test_acc.append(metric["acc"])

        print(f'[Epoch {cb_params.cur_epoch_num}] '
              f'loss={loss:.4f}, test_acc={metric["acc"]:.4f}')
```

运行训练程序后，得到原始模型训练过程的 Loss 和 Accuracy 曲线如图 3，显然发生了**非常剧烈的波动**！运行到 150 轮时 Accuracy 为 **0.8283**，但是肉眼可见的，曲线还在上升，**没有收敛**！收敛速度仍有待提高。

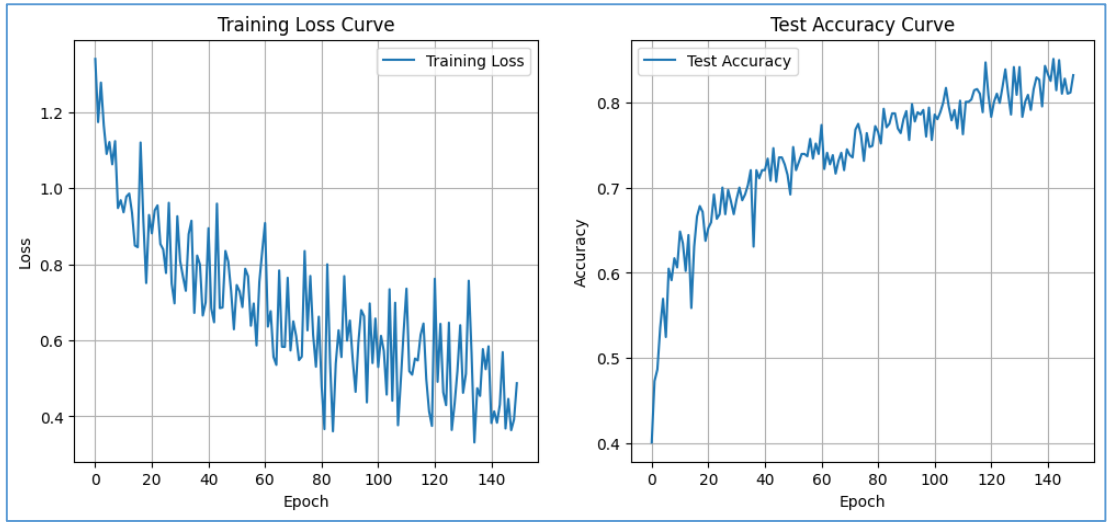


图 3 原始模型训练过程的 Loss 和 Accuracy 曲线

3.1.2 网络结构改进与再训练

原始网络结构比较接近 AlexNet（或 VGG 的简化版），网络“深、参数量大、无归一化、无正则”，因此在训练时非常容易出现：梯度爆炸/不稳定、Loss 曲线大幅震荡、学习率敏感。具体**结构性风险**如表 1 所示。

表 1 原始模型存在的具体结构性风险及后果

原始网络的结构性风险	可能导致的后果
• 所有卷积后均无 BatchNorm	→ 特征分布漂移大，梯度方差大
• 学习率($\sigma=0.01$)初始化过小	→ 对深网络不友好，早期梯度弱/不稳定
• 无 Dropout	→ 前期 batch 波动会直接放大到 loss
• 卷积层 conv3/conv4 后无激活函数 ReLU，直接接了池化层	→ 纯线性变换，大大降低了网络提取 非线性特征 的能力，严重限制 accuracy

根据以上风险判断，对网络结构作如下调整（代码段 2）。原始模型训练程序中，Adam 在小 batch 下噪声大，是造成 loss 曲线**严重震荡**的原因之一。于是更改优化器为 **RMSProp**（代码段 3），理论上更新**平滑**，可让曲线**变稳**。

代码段 2：改进后 CNN 网络（BatchNorm + Dropout，激活函数改 LeakyReLU）

```
class Identification_Net(nn.Cell):
    # .....先前代码不变.....
    # 卷积层 + BatchNorm
    self.conv1 = nn.Conv2d(self.channel, 32, kernel_size=5, stride=1, padding=0,
                           has_bias=True, pad_mode="same",
                           weight_init=TruncatedNormal(sigma=trun_sigma), bias_init='zeros')
    self.bn1 = nn.BatchNorm2d(32)
    # ..... conv2、conv3、conv4（同理，略） .....

```

```

# 激活函数由 ReLU 改用 LeakyReLU
self.relu = nn.LeakyReLU()
# 池化层
self.max_pool2d = nn.MaxPool2d(kernel_size=2, stride=2, pad_mode="valid")
# ..... 全连接部分（未修改，略） .....
# 全连接部分：减小全连接层规模（1024 改 512）
self.fc1 = nn.Dense(6 * 6 * 128, 512, ..... )
self.fc2 = nn.Dense(512, 256, ..... )
self.fc3 = nn.Dense(256, self.num_class, ..... )
# Dropout（注意：MindSpore 1.7 版本用的参数叫 keep_prob 而非 p）
self.dropout = nn.Dropout(keep_prob=0.2)

```

代码段 3：改进模型训练过程中的优化器（Adam 改成 RMSProp）

```

net_opt = nn.RMSProp (params=group_params, learning_rate=cfg.lr,
                      decay=0.9, momentum=0.0, epsilon=1e-10,
                      weight_decay=1e-4, centered=False)

```

RMSProp 比 Adam 更稳定，但更新幅度小。learning_rate（学习率）保持 $1e-4$ ；decay（梯度平方的滑动平均系数）默认 0.9 最稳；目标是减少震荡，所以 momentum 保持 0.0，更平稳；weight_decay 为每次参数更新时对权重施加的额外惩罚，使权重趋向更小、更平滑，尝试设置 $1e-4$ （或许可以改善过拟合？）。

执行改进模型的训练程序，最后 5 个 epoch 的 Loss 和 Accuracy 如图 4，可见 Accuracy 在 0.79 左右收敛！改进模型训练过程两条曲线如图 5。

```

epoch: 146 step: 22, loss is 0.5431495904922485
[Epoch 146] loss=0.5431, test_acc=0.7916
epoch: 147 step: 22, loss is 0.527357280254364
[Epoch 147] loss=0.5274, test_acc=0.7902
epoch: 148 step: 22, loss is 0.4436393678188324
[Epoch 148] loss=0.4436, test_acc=0.7738
epoch: 149 step: 22, loss is 0.46474048495292664
[Epoch 149] loss=0.4647, test_acc=0.7970
epoch: 150 step: 22, loss is 0.4519648253917694
[Epoch 150] loss=0.4520, test_acc=0.8188

```

图 4 改进模型训练过程中最后 5 个 epoch 的 Loss 和 Accuracy

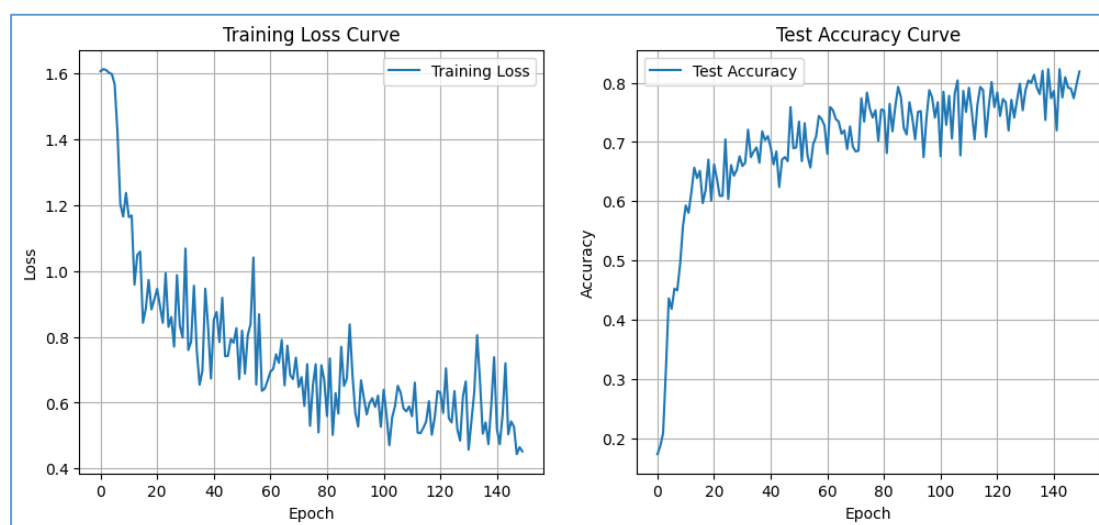
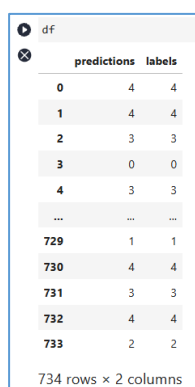


图 5 改进模型训练过程的 Loss 和 Accuracy 曲线

3.1.3 改进模型预测与评估

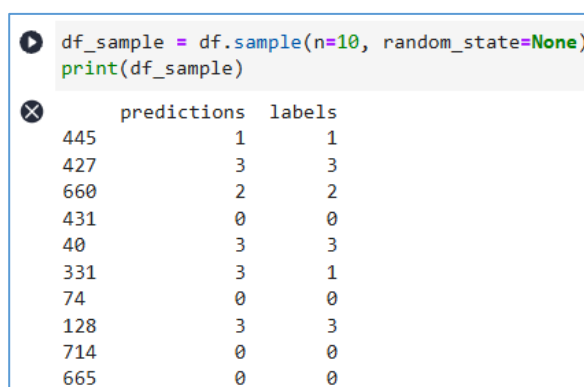
模型改进后，继续完成模型评估，预测结果输出为 result.csv（图 6），以下展示测试集个别图像的预测结果（图 7）。



	predictions	labels
0	4	4
1	4	4
2	3	3
3	0	0
4	3	3
...	...	...
729	1	1
730	4	4
731	3	3
732	4	4
733	2	2

734 rows x 2 columns

图 6 预测结果 df（部分）



```
df_sample = df.sample(n=10, random_state=None)
print(df_sample)
```

	predictions	labels
445	1	1
427	3	3
660	2	2
431	0	0
40	3	3
331	3	1
74	0	0
128	3	3
714	0	0
665	0	0

图 7 随机抽取 10 个预测结果

为了更清晰地分辨分类的准确程度，编写程序绘制混淆矩阵（代码段 4），运行该程序可直接得到分类预测的混淆矩阵（图 8）。

代码段 4：利用 scikit-learn、seaborn 绘制混淆矩阵

```
from sklearn.metrics import confusion_matrix
import seaborn as sns # 导入热力图模块
cm = confusion_matrix(labels, predictions) # 借用 sklearn 工具生成混淆矩阵
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=[class_names[i] for i in range(num_classes)],
            yticklabels=[class_names[i] for i in range(num_classes)])
# .....利用 matplotlib 模块展示图片（略）.....
```

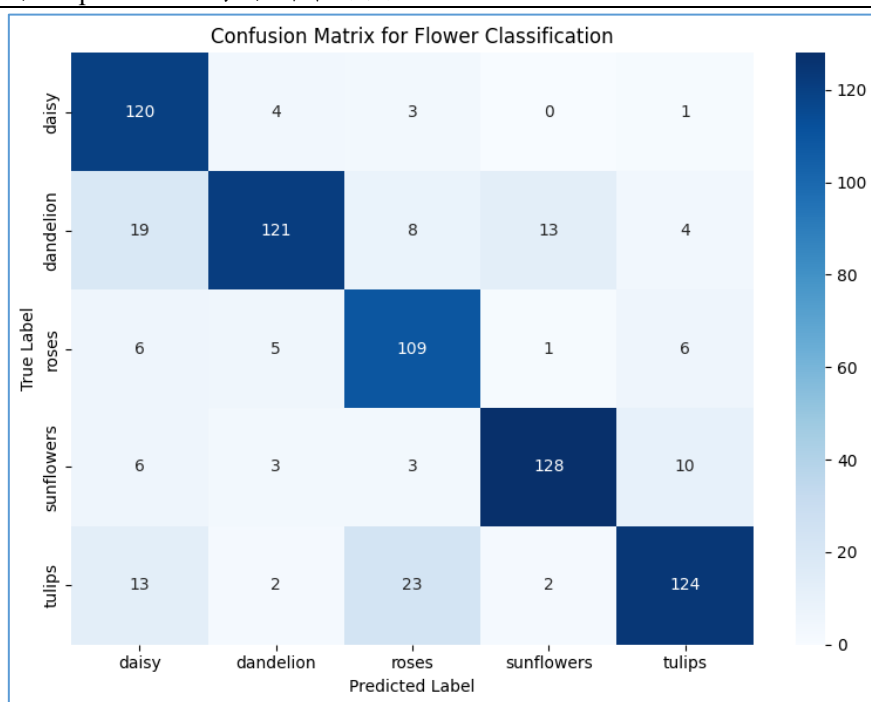


图 8 改进模型进行分类预测的混淆矩阵

如混淆矩阵（图 8）所示，该模型较为准确地分类出 daisy、dandelion、roses、sunflowers、tulips 五种不同的花卉，且斜对角线元素权重较高（热力图较深），说明改进模型的准确率较高，符合实验要求。

3.2 任务二

分别输出高级特征与中级特征的 t-sne 可视化图，并附上简单的解释。

t-SNE (t-DistributedStochastic Neighbor Embedding, T 分布随机近邻嵌入) 是一种可以把高维数据降到二维或三维的降维技术。我们使用 t-sne 来可视化提取到的特征的分布。

3.2.1 模型结构及特征层级判断

```
(0): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(1): ReLU()
(2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
(3): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(4): ReLU()
(5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
(6): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
(8): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(9): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
```

图 9 改进后的模型结构描述

根据模型结构（如图 9）判断得到高级、中级特征所代表的层级（如表 2），分别赋值给 index。执行程序分别得到高级特征 t-sne 可视化图（图 10）与中级特征 t-sne 可视化图（图 11）。

表 2 模型层级及其对应索引

特征层级	对应 Conv 层 index	说明
低级特征	0	颜色、边缘、角点
中级特征	3、6	纹理、局部结构、花瓣细节
高级特征	8	花朵语义结构、类别相关抽象特征

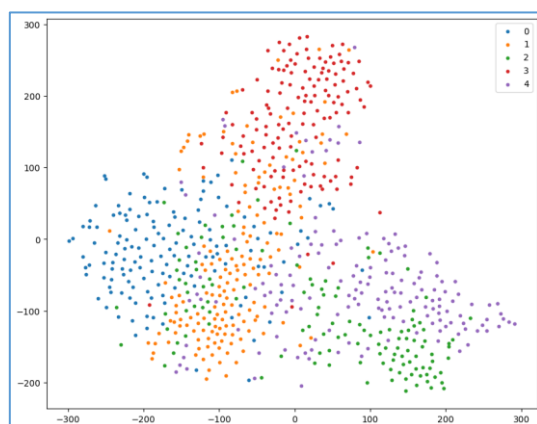


图 10 高级特征 t-sne 可视化图

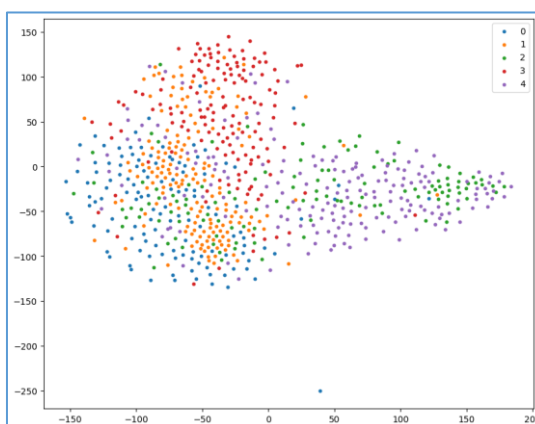


图 11 中级特征 t-sne 可视化图

3.2.2 差异对比及原因分析

对于高级特征(index=8), 可见明显的**聚类 (Clustering)** 现象。不同颜色 (代表不同类别的花卉) 的数据点形成了彼此分离的“岛屿”。类内距离较小, 类间距离较大。即数据表现出高度的**线性可分性**。

对于中级特征(index=6), 数据点呈现**混杂 (Entangled)** 状态。虽然能看到一些“聚集”的趋势 (例如橙色在左上, 紫色在右下), 但不同类别之间没有清晰边界, 大量的点相互重叠、交织。即数据表现为模糊的分布, 类别区分度低。

这种差异归因于 (或者说是恰恰体现出了) 卷积神经网络 (CNN) 处理图像信息时的**层级抽象特性**:

(1) 中级特征 (index=6) —— “看局部”

① **特征共享**: 不同的花卉往往拥有相似的局部特征。例如, 玫瑰和郁金香可能都有红色的花瓣纹理, 蒲公英和向日葵可能都有绿色的茎叶边缘。

② **语义未对齐**: 在这一层, 神经网络关注的是“这是什么形状/纹理”, 而不是“这是什么花”。因为不同种类的花在局部纹理上非常相似, 所以 t-SNE 将它们映射到二维空间时, 它们依然混在一起。

(2) 高级特征 (index=8) —— “看整体”

① **语义整合**: 随着网络层数加深, 模型将中级的纹理和形状组合成了完整的“对象”概念。模型已经学会了忽略无关的细节 (如背景杂乱), 专注于区分不同花卉的关键特征 (如整体轮廓、特定的花蕊结构)。

② **目标驱动**: CNN 训练的最终目的是分类 (降低 Loss)。在靠近输出层的深层 (index=8), 网络被迫将特征空间进行变换, 使得不同类别的数据尽可能分开, 以便最后的分类器 (全连接层) 能划出一道清晰的界限。因此, t-SNE 展现出了清晰的聚类效果。

3.3 任务三

在原来模型结构的基础上, 参考下图 12, 增加残差连接。附上修改后的模型构建代码与结果 Acc。

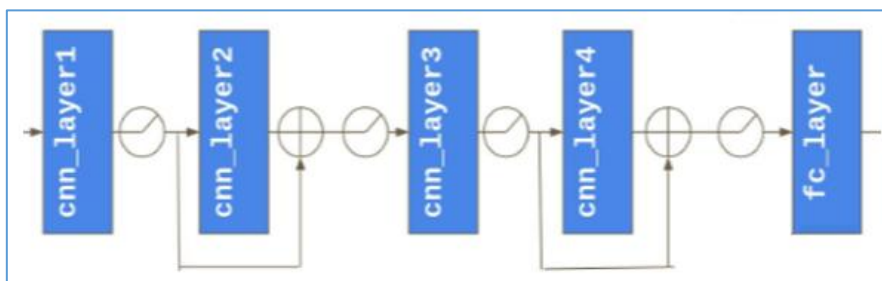


图 12 增加残差连接后模型示意图

根据图 12, 修改后的模型构建代码如下 (代码段 5)。

代码段 5: 增加残差连接后的网络模型 (根据最初 ver 原始模型修改)

```
import mindspore.nn as nn
from mindspore.common.initializer import TruncatedNormal
```

```

class Identification_Net(nn.Cell):
    def __init__(self, num_class=5, channel=3, trun_sigma=0.01):
        # ..... (该部分不变, 略) .....
        # 权重初始化配置
        weight_init = TruncatedNormal(sigma=trun_sigma)
        # --- Layer 1 ---
        self.conv1 = nn.Conv2d(.....)
        self.bn1 = nn.BatchNorm2d(32) # 增加 BN 层
        self.relu = nn.ReLU()
        self.max_pool2d = nn.MaxPool2d(kernel_size=2, stride=2, pad_mode="valid")

        # --- Layer 2 (残差块 1 主路径) ---
        self.conv2 = nn.Conv2d(.....)
        self.bn2 = nn.BatchNorm2d(64)

        # ----- 残差连接 1 (shortcut) -----
        # 此时主路径经过了 Conv2(变通道)和 Pool(变尺寸), shortcut 需要匹配
        # 输入 32 通道 -> 输出 64 通道, Stride=2 匹配池化
        self.downsample1 = nn.SequentialCell([
            nn.Conv2d(32, 64, kernel_size=1, stride=2, pad_mode="same",
                      has_bias=False, weight_init=weight_init),
            nn.BatchNorm2d(64)
        ])

        # --- Layer 3 ---
        self.conv3 = nn.Conv2d(.....)
        self.bn3 = nn.BatchNorm2d(128)

        # --- Layer 4 (残差块 2 主路径) ---
        self.conv4 = nn.Conv2d(.....)
        self.bn4 = nn.BatchNorm2d(128)

        # ----- 残差连接 2 (shortcut) -----
        # 主路径经过 Pool(变尺寸), shortcut 需要匹配
        # 输入 128 通道 -> 输出 128 通道, Stride=2 匹配池化
        self.downsample2 = nn.SequentialCell([
            nn.Conv2d(128, 128, kernel_size=1, stride=2, pad_mode="same",
                      has_bias=False, weight_init=weight_init),
            nn.BatchNorm2d(128)
        ])

        self.flatten = nn.Flatten()
        # ..... (全连接层保持不变, 略) .....

```



```
def construct(self, x):
```

```
# ===== Block 1 Start =====
# Layer 1
x = self.conv1(x)          # ----- ①
x = self.bn1(x)            # ----- ②
x = self.relu(x)           # ----- ③
x = self.max_pool2d(x)     # ----- ④
# 保存 Layer1 的输出作为残差项 identity
identity1 = x

# Layer 2
# (略, 同上①~④)
# 处理残差项 (下采样以匹配 x 的维度)
identity1 = self.downsample1(identity1)    # downsample 功能见补充
# 残差相加
x = x + identity1
x = self.relu(x)
# ===== Block 1 End =====
```

```
# ===== Block 2 Start =====
# Layer 3
# (略, 同上①~④)
# 保存 Layer3 的输出作为残差项 identity
identity2 = x

# Layer 4
# (略, 同上①~④)
# 处理残差项
identity2 = self.downsample2(identity2)    # downsample 功能见补充

# 残差相加
x = x + identity2
x = self.relu(x)
# ===== Block 2 End =====
```

```
# 全连接层 (保持不变)
x = self.flatten(x)
x = self.fc1(x)
x = self.relu(x)
x = self.fc2(x)
x = self.relu(x)
x = self.fc3(x)
return x
```

【补充】downsample 模块适配残差项与主路径的通道数、尺寸一致性，确保了残差相加的有效性，这也是残差连接能充分发挥作用的关键技术细节。

在原始模型基础上增加残差连接后，训练结果如图 13，准确率（Accuracy）达到 0.8542。可见残差连接能够缓解深层网络的梯度衰减/消失问题、提升训练稳定性与收敛速度、增强模型泛化能力。

```
===== Starting Training =====  
epoch: 10 step: 45, loss is 0.8705975413322449  
epoch: 20 step: 45, loss is 0.8179661631584167  
epoch: 30 step: 45, loss is 0.6585023999214172  
epoch: 40 step: 45, loss is 0.6862497925758362  
epoch: 50 step: 45, loss is 0.4004429280757904  
epoch: 60 step: 45, loss is 0.41251540184020996  
epoch: 70 step: 45, loss is 0.6122412085533142  
epoch: 80 step: 45, loss is 0.6037088632583618  
epoch: 90 step: 45, loss is 0.3898111581802368  
epoch: 100 step: 45, loss is 0.5511866211891174  
epoch: 110 step: 45, loss is 0.4252385199069977  
epoch: 120 step: 45, loss is 0.4264658987522125  
epoch: 130 step: 45, loss is 0.5059665441513062  
epoch: 140 step: 45, loss is 0.35090371966362  
epoch: 150 step: 45, loss is 0.19484806060791016  
{'acc': 0.8542234332425068}
```

图 13 增加残差连接后的训练结果

四、实验结论与讨论

4.1 实验结论

本次基于 MindSpore 的花卉识别实验，通过模型优化与结构改进达成了核心目标，主要结论如下：

（1）掌握了 CNN 网络基础及 MindSpore 开发流程，成功实现了五种花卉的分类任务，验证了 MindSpore 在图像分类场景的可用性。

（2）原始模型存在无归一化、无正则、激活函数设计不合理等问题，导致训练震荡且未收敛，经 BatchNorm、Dropout、LeakyReLU 及优化器替换（Adam → RMSProp）后，模型稳定收敛，测试准确率提高。

（3）t-SNE 可视化验证了 CNN 的层级抽象特性：中级特征（纹理、局部结构）呈现混杂分布，高级特征（语义结构、类别抽象特征）聚类清晰，类间区分度显著提升。

（4）增加残差连接后，模型解决了深层网络梯度衰减问题，进一步提升了特征提取能力，最终测试准确率大大提升。

4.2 思考题

1. Softmax 函数是用于多分类任务的，如果是处理二分类任务我们用什么函数呢？

Sigmoid 函数（Logistic 函数）：能将任意实数映射到 0~1 的区间，刚好可以表示“样本属于某一类”的概率，另一类的概率直接用“1-该概率”计算即可，更简洁高效。

2. 请结合其他函数的特点，思考 ReLU 函数的优缺点？

优点：

(1) ReLU 的收敛速度比 sigmoid 和 tanh 快；（梯度不会饱和，解决了梯度消失问题）

(2) 计算复杂度低，不需要进行指数运算；

(3) 适合用于后向传播。

缺点：

(1) ReLU 的输出不是 zero-centered；

(2) Dead ReLU Problem（神经元坏死现象）：某些神经元可能永远不会被激活，导致相应参数永远不会被更新（在负数部分，梯度为 0）。产生这种现象的两个原因：参数初始化问题；learning rate 太高导致在训练过程中参数更新太大。

解决方法：采用 Xavier 初始化方法，以及避免将 learning rate 设置太大或使用 adagrad 等自动调节 learning rate 的算法。

(3) ReLU 不会对数据做幅度压缩，所以数据的幅度会随着模型层数的增加不断扩张。

3. Flatten 层用来将输入“压平”，即把多维的输入一维化，我们为什么要做这部分操作呢？

核心：将所有图片的像素值转化为一列，映射下一层的神经元。

(1) 衔接网络结构：卷积层、池化层的输出是多维特征图（ $H \times W \times C$ ，即高度 $\times$ 宽度 $\times$ 通道数），而**全连接层的输入要求为一维向量**（每个神经元需与前一层所有特征点连接）。Flatten 层通过按序展开特征图的所有元素，搭建了“特征提取（卷积层）”与“分类决策（全连接层）”的关键桥梁，如实验中卷积层输出的（ $6 \times 6 \times 128$ ）特征图，经 Flatten 后转化为 4608 维向量，才能送入 fc1 层进行后续运算。

(2) 保留特征完整性：Flatten 层仅改变特征的维度形态，不丢失任何像素或通道的特征信息，确保卷积层提取的花卉空间特征（如花瓣纹理、花蕊结构）能**完整传递给全连接层**，为分类判断提供全面依据。

(3) 适配分类器逻辑：全连接层的核心作用是通过权重矩阵对特征进行线性组合与非线性映射，最终输出类别概率。一维向量形式**能让权重矩阵与特征向量直接进行矩阵运算**，符合分类器的数学建模逻辑，是实现花卉类别决策的必要前提。